有趣的小游戏

先运行一下游戏，根据输入移动，然后吃金币得分，不难猜测会有一个 while 循环，不断进行读取输入然后加密

```
v11[24] = 555485517;
v11[25] = 2011874035;
v11[26] = 351303366;
v11[27] = 715586549;
v11[28] = 0x2DEB36E4;
v11[29] = 0x669AA5C8;
sub_460E20((__int64)&v12);
v7[0] = (__int64)v11;
v7[1] = 30i64;
sub_48F790((__int64)v10, v7, (__int64)&v12);
sub_460E90(&v12);
sub_48F6A0(v13, (__int64)v10);
sub_41D970(v9, v13);
sub_48F840(v13);
```

While 循环前主要是初始化 v9，v9 可以看作一个结构体，其成员包括了 p，c,e 的坐标，分数，地图信息等

注意 v11 被写入 v9+56 的位置

```
v13 = a1;
v9 = sub_48EC00(a1);
v8 = sub_48EBD0(v13);
while ( (unsigned __int8)sub_420110(&v9, &v8) )
{
  v12 = sub_420FF0(&v9);
  v11 = v12;
  v7 = sub_42A210(v12);
  v6 = sub_42A1B0(v11);
  while ( (unsigned __int8)sub_4200D0(&v7, &v6) )
  {
    v10 = *(_BYTE *)sub_420FB0(&v7);
    sub_4A2B60(&qword_4A9880, (unsigned int)v10);
    sub_41EFD0(&v7);
  }
  ((void (__fastcall *)(void *))sub_4A0630)(&qword_4A9880);
  sub_41F030(&v9);
}
v1 = sub_4A2AA0(&qword_4A9880, &unk_4AA015);
v2 = sub_469320(v1, *(unsigned int *)(a1 + 52));
v3 = sub_4A2AA0(v2, &unk_4AA022);
v4 = sub_469320(v3, *(unsigned int *)(a1 + 48));
return ((__int64 (__fastcall *)(__int64))sub_4A0630)(v4);
```

41D820 打印地图和分数

调试发现，v9+52 储存当前分数，+48 储存总分数（0x3E8==1000）

```
sub_401550();
sub_41D820(v9);
v3 = sub_4A2AA0(&qword_4A9880, &unk_4AA028);
((void (__fastcall *)(__int64))sub_4A0630)(v3);
sub_4A2AA0(&qword_4A9880, &unk_4AA050);
sub_4A3A70(&unk_4A9520, &v8);
if ( (unsigned __int8)sub_41D620(v9, (unsigned int)v8) ≠ 1 )
{
  v4 = sub_4A2AA0(&qword_4A9880, &unk_4AA07E);
  ((void (__fastcall *)(__int64))sub_4A0630)(v4);
  sub_4656C0(&unk_4A9520);
  sub_464DE0(&unk_4A9520);
```

循环内 4a3a70 读取输入，41D620 进行处理，我们跟进

```
y = a1[6];
x = a1[7];
if ( a2 == 'd' )
{
  ++y;
}
else if ( a2 > 100 )
{
  if ( a2 == 115 )
  {
    ++x;
  }
  else
  {
    if ( a2 ≠ 119 )
      return 0i64;
    --x;
  }
}
else
{
  if ( a2 ≠ 97 )
    return 0i64;
```

前面根据输入改变 x,y 坐标

```
if ( !v4 )
  return 0i64;
v5 = sub_48ED00(a1, x);
if ( *(_BYTE *)sub_48F410((_QWORD *)v5, y) == '#' )
  return 0i64;
sub_41D580((__int64)a1, a2);
v6 = (int)a1[6];
v7 = sub_48ED00(a1, (int)a1[7]);
*(_BYTE *)sub_48F410((_QWORD *)v7, v6) = ' ';
a1[6] = y;
a1[7] = x;
v8 = (int)a1[6];
v9 = sub_48ED00(a1, (int)a1[7]);
```

中间检测了是否撞墙，然后会把当前位置变为空格

```
if ( *(_BYTE *)sub_48F410((_QWORD *)v9, v8) == 'C' )
{
  ++a1[13];
  sub_41D580((__int64)a1, 100u);
  if ( a1[13] < a1[12] )
    sub_41D3C0(a1);
}
v10 = (int)a1[6];
v11 = sub_48ED00(a1, (int)a1[7]);
*(_BYTE *)sub_48F410((_QWORD *)v11, v10) = 'P';
return 1i64;
}
```

根据是否吃到金币进行操作，最后移动 p 的位置

跟进这里的 41d580

```
result = a2;
if ( (_BYTE)a2 == 'd' || (_BYTE)a2 == 'w' )
{
  v3 = sub_42A2D0(a1 + 56);
  return sub_40165D(a1 + 56, (unsigned int)-v3, a1 + 80);
}
else if ( (_BYTE)a2 == 97 || (_BYTE)a2 == 115 )
{
  v4 = sub_42A2D0(a1 + 56);
  return sub_40165D(a1 + 56, v4, a1 + 80);
}
return result;
```

+ - v4 的结构，很想加解密了，跟进 40165d

```
if ( a2 ≤ 1 )
{
  if ( a2 < -1 )
  {
    v4 = (void (__fastcall *)(__int64, _QWORD, __int64))sub_41C090("file2.txt");
    if ( v4 )
    {
      v3 = sub_48F610(a1);
      v4(v3, (unsigned int)a2, a3);
      VirtualFree(v4, 0i64, 0x8000u);
    }
  }
}
```

根据 v4 的值读取不同的 file，这里把读取的内容作为函数执行了，步过到执行 v4 时，步进，把函数内容复制粘贴下来，保存后丢 ida

```
v6 = 52 / v11 + 6;
v8 = 0;
v9 = a1[v11 - 1];
do
{
  v8 -= 1640531527;
  v5 = (v8 >> 2) & 3;
  for ( i = 0; i < v11 - 1; ++i )
  {
    v10 = a1[i + 1];
    v1 = a1[i]
       + (((v9 ^ *(_DWORD *)(v12 + 4i64 * (v5 ^ i & 3))) + (v10 ^ v8)) ^ (((16 * v9) ^ (v10 >> 3))
                                                                         + ((4 * v10) ^ (v9 >> 5))));
    a1[i] = v1;
    v9 = v1;
  }
  v2 = v11 - 1;
  v3 = a1[v2]
     + (((v9 ^ *(_DWORD *)(v12 + 4i64 * (v5 ^ i & 3))) + (*a1 ^ v8)) ^ (((16 * v9) ^ (*a1 >> 3))
                                                                       + ((4 * *a1) ^ (v9 >> 5))));
  a1[v2] = v3;
  v9 = v3;
  result = (unsigned int)(v6 - 1);
  v6 = result;
}
while ( (_DWORD)result );
return result;
```

明显的 xxtea 解密特征，不难猜到另一个文件就是加密了

那么逻辑很清晰了，每次输入都会执行一次加密操作，明文就是 v9+56,密钥是 v9+80，扣下来就行，如果吃到金币，就多执行一次。由于不知道吃到金币会走多少步（路径不唯一），因此只能多次解密，知道出现特征（ISCC{}）

EXP:

```
#include <stdio.h>

#include <stdint.h>
```

```
#include <string.h>

void xxtea_decrypt(uint32_t *v, int n, const uint32_t key[4]) {
    uint32_t delta = 0x9e3779b9;
    int r = 6 + 52 / n;
    uint32_t total = delta * r;
    uint32_t v0, v1;
    int i, j;

    for (i = 0; i < r; i++) {
        uint32_t e = (total >> 2) & 3;
        for (j = n - 1; j > 0; j--) {
            v1 = v[j - 1];
            uint32_t part1 = (v1 >> 5) ^ (v[j] << 2);
            uint32_t part2 = (v[j] >> 3) ^ (v1 << 4);
            uint32_t part3 = (total ^ v[j]) + (key[(j & 3) ^ e] ^ v1);
            v[j] -= (part1 + part2) ^ part3;
        }
        v0 = v[n - 1];
        uint32_t part1 = (v0 >> 5) ^ (v[0] << 2);
        uint32_t part2 = (v[0] >> 3) ^ (v0 << 4);
        uint32_t part3 = (total ^ v[0]) + (key[(0 & 3) ^ e] ^ v0);
```

```
            v[0] -= (part1 + part2) ^ part3;
            total -= delta;
        }
}

int main() {
        uint32_t v[] = {
            0x3d0db055, 0x6aa21e1e, 0x5bc2f195, 0x372f36ac,
0xb3c606be,
            0xce1077e2, 0x69fc40b1, 0x8f19d215, 0xf9a42897,
0x960fe985,
            0xce0f9ef1, 0x6aaec9ef, 0x00d97f91, 0x4f43d749,
0x73ad8ec2,
            0xd30e3dd4, 0xefa46ea4, 0x13b0ebe7, 0xcaef3614,
0xdb9dcaf7,
            0x97731560, 0x05b9a135, 0xa60e32c6, 0x0818b8c4,
0xb6133c72,
            0x92979bfc, 0xf23a75f1, 0x09c6b17c, 0x8191d18e,
0x80844b55
        };
        uint32_t key[] = {0x12345678, 0x9ABCDEF0, 0xFEDCBA98,
0x76543210};
```

```
int length = sizeof(v) / sizeof(v[0]);
int round;

for (round = 0; round < 20000; round++) {
    xxtea_decrypt(v, length, key);

    // Convert to little-endian bytes
    unsigned char bytes[30 * 4];
    for (int i = 0; i < 30; i++) {
        bytes[i * 4 + 0] = (v[i] >> 0) & 0xFF;
        bytes[i * 4 + 1] = (v[i] >> 8) & 0xFF;
        bytes[i * 4 + 2] = (v[i] >> 16) & 0xFF;
        bytes[i * 4 + 3] = (v[i] >> 24) & 0xFF;
    }

    // Check for printable characters
    char buffer[30 * 4 + 1];
    int pos = 0;
    for (int i = 0; i < 30 * 4; i++) {
        if (bytes[i] >= 0x20 && bytes[i] <= 0x7E) {
            buffer[pos++] = bytes[i];
        } else {
```

```
                if (pos > 0 && buffer[pos - 1] != ' ') {
                    buffer[pos++] = ' ';
                }
            }
        }
        buffer[pos] = '\0';

        // Search for flag pattern
        char *start = strstr(buffer, "ISCC{");
        if (start != NULL) {
            char *end = strchr(start, '}');
            if (end != NULL && end > start) {
                printf("Found at round %d: %.*s}\n", round + 1,
(int)(end - start + 1), start);
                break; // Remove if multiple flags possible
            }
        }
    }

    return 0;
}
```